

AMAL JYOTHI COLLEGE OF ENGINEERING KANJIRAPPALLY

B. TECH - COMPUTER SCIENCE AND ENGINEERING

BTCS2023-27-S6: 23CST362-PROGRAMMING IN PYTHON-Project (Batch – I)

Modern Marks Calculator GUI (Pygame)

Topic Title	Modern Marks Calculator GUI (Pygame)
Student Name	Noel B Augustine
Register Number	AJC23CS163
Class/Section	CSE C

Abstract

Calculating student grades manually can be time-consuming and prone to human error. This project implements a modern, automated desktop application that calculates a student's total marks, percentage, and corresponding letter grade based on their input scores. Unlike standard GUI frameworks, this application utilizes Pygame, a powerful multimedia library typically used for game development, to create a custom-built graphical user interface from scratch. This approach demonstrates deep understanding of coordinate systems, rendering loops, and event-driven programming. The application features built-in input validation to prevent crashes from non-numeric or out-of-bounds data, responsive mouse and keyboard controls, and dynamic color-coded feedback. Ultimately, this project demonstrates a complete, simple, and stable practical application of Python GUI development beyond traditional frameworks.

Problem Description

Educational institutions and individuals frequently need to calculate aggregate totals, percentages, and categorical letter grades from raw test scores. Performing these calculations manually is inefficient. The objective of this project is to develop a Python-based interactive desktop application that streamlines this process. The system must accept inputs for three subjects (Math, Science, and English), mathematically process these inputs to generate a total and percentage, and apply conditional logic to determine a final grade. Furthermore, the application must handle user errors gracefully, ensuring that invalid inputs (like text or scores exceeding the maximum limit) do not cause runtime failures. The unique challenge of this project is building the entire user interface from scratch using Pygame's low-level rendering capabilities rather than relying on pre-built GUI frameworks.

Libraries & Tools Used (with purpose)

- **Pygame (Beyond Syllabus Library):** Used to construct the graphical user interface from scratch. Pygame is a multimedia library that provides low-level access to rendering graphics, handling events, managing fonts, and processing keyboard/mouse input. Unlike standard GUI frameworks that provide pre-built widgets, Pygame requires manual implementation of all UI components, demonstrating mastery of coordinate systems, event loops, and custom rendering.
- **Pygame.font (TrueType Font Rendering):** Used to render professional typography by loading system fonts (Segoe UI) and rendering text at various sizes for labels, input fields, and output displays.
- **Pygame.draw (Graphics Primitives):** Used to manually draw UI elements such as rectangles for input fields, buttons, and containers. All visual elements are constructed using geometric primitives rather than pre-built widgets.

System Design / Workflow

- **Initialization Phase:** The Pygame library initializes a 350×570 pixel window with custom fonts loaded from the system. State variables are initialized to track active input field, user input strings, and calculation results.
- **Event Loop Phase:** The application runs continuously at 60 FPS, listening for mouse clicks (to select input fields or trigger calculation), keyboard input (to enter numeric data), and special keys (TAB to cycle fields, ENTER to calculate, BACKSPACE to delete).
- **Input Validation Phase:** The system filters keystrokes in real-time to accept only numeric characters and decimal points. Before calculation, inputs are validated to ensure they are non-empty and within the 0-100 range.
- **Processing Phase:** Calculates the sum of three subjects, computes percentage based on maximum 300 marks, and evaluates percentage against a predefined grading scale (A to F) using conditional statements.
- **Rendering Phase:** The screen is cleared and redrawn every frame with updated colors, text, and visual feedback. Input fields highlight when active, buttons respond to hover states, and the grade displays in color-coded format.

Implementation Details

The application is built entirely in Python using Pygame for rendering. The UI is initialized by creating a `pygame.display` window with specified dimensions. Unlike traditional GUI frameworks, all visual elements are manually drawn using `pygame.draw` functions for rectangles and `pygame.font` for text rendering.

Input fields are represented as `pygame.Rect` objects that define clickable regions. The application uses collision detection (`rect.collidepoint()`) to determine which input field is

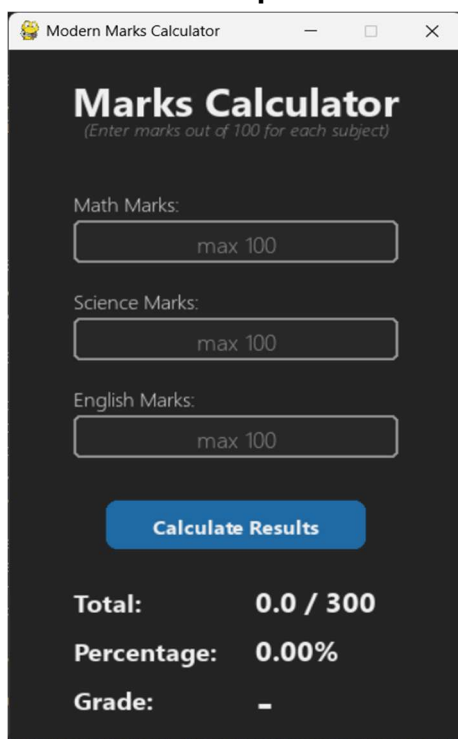
clicked. The active input field is tracked via an `active_input` variable, and keyboard events are routed to update the corresponding input string.

The core processing logic is housed within the `calculate()` function, which attempts to convert string inputs to floats using try-except blocks. Input validation uses Python's `any()` function to iterate through scores and verify they fall within bounds. The grading logic employs if-elif-else chains to assign both a letter grade and a hex color code for visual feedback.

The rendering loop runs at 60 FPS using `pygame.time.Clock()`. Each frame, the screen is filled with the background color, all UI elements are redrawn with updated states (active field highlighting, calculated results), and `pygame.display.flip()` updates the display. This continuous rendering creates smooth visual feedback for user interactions.

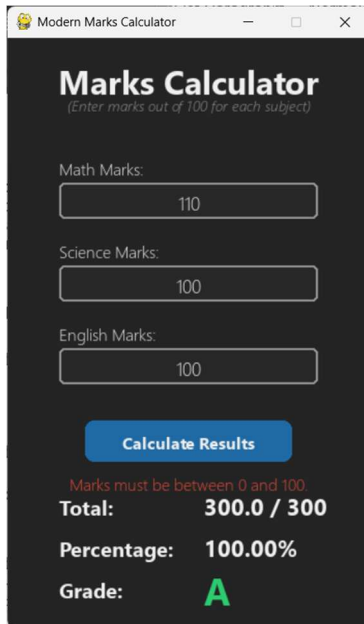
Results & Output (with screenshots)

- The application successfully calculates and displays grades while mitigating user input errors through real-time validation and error handling.
- **Screenshot 1: Input / Interface / Start screen**



Description: The initial custom-built Pygame interface awaiting user input, showing the dark theme, professionally rendered input fields with placeholders, and the Calculate Results button.

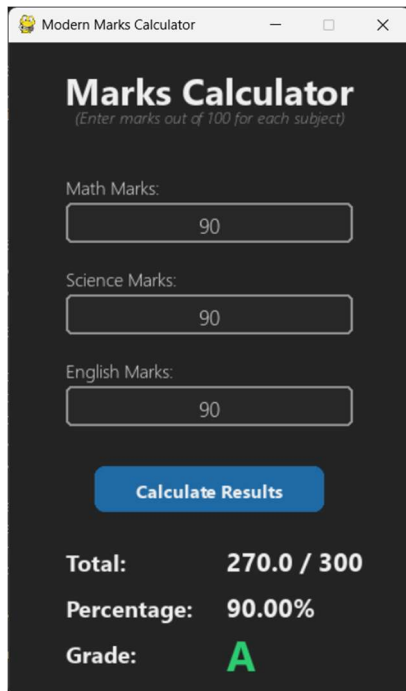
- **Screenshot 2: Input Validation / Error Handling**



The screenshot shows a web application window titled "Modern Marks Calculator". The main heading is "Marks Calculator" with a subtitle "(Enter marks out of 100 for each subject)". There are three input fields: "Math Marks:" with the value "110", "Science Marks:" with the value "100", and "English Marks:" with the value "100". Below these fields is a blue button labeled "Calculate Results". At the bottom of the window, there is a red error message: "Marks must be between 0 and 100". Below the error message, the results are displayed: "Total: 300.0 / 300", "Percentage: 100.00%", and "Grade: A".

When a mark exceeds 100, the application displays an error message in red text at the bottom of the window, preventing invalid calculations. Description: The validation logic intercepting out-of-bounds input to prevent calculation errors, demonstrating graceful error handling.

- **Screenshot 3: Final output / result**



Description: The final processed output displaying the total marks (270.0 / 300), percentage (90.00%), and dynamic color-coded letter grade (A in green), demonstrating the complete calculation workflow with professional visual presentation.

Conclusion & Scope for Mini Project Extension

This project successfully demonstrates a stable, standalone implementation of a modern GUI using Pygame, showcasing skills beyond traditional GUI frameworks. The program fulfills all mathematical processing and validation requirements without runtime errors, while demonstrating mastery of low-level graphics rendering, event handling, and custom UI development.

Scope for Extension:

This individual mini-project can easily serve as a foundational module for a larger Mini Project. The current logic can be extended into a comprehensive "Student Management System." Future iterations could integrate an SQLite database to permanently store calculated records, add data visualization using Matplotlib charts embedded in the Pygame interface to track student performance over time, implement batch processing via CSV file uploads, add export functionality to generate PDF reports, or create a multi-user system with authentication. The custom-built UI foundation provides flexibility for advanced features not easily achievable with standard GUI frameworks. This ensures continuity in problem understanding and design for future development phases.

References

- Python Software Foundation. (n.d.). Python Language Reference. Retrieved from python.org
- Pygame Development Team. (n.d.). Pygame Documentation. Retrieved from pygame.org/docs/

Appendix: Source Code

```
import pygame
import sys

pygame.init()
WIDTH, HEIGHT = 350, 570
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Modern Marks Calculator")

# Colors
BG_COLOR = (36, 36, 36)
TEXT_COLOR = (240, 240, 240)
GRAY = (150, 150, 150)
INPUT_BG_ACTIVE = (50, 50, 50)
INPUT_BG_INACTIVE = (40, 40, 40)
BUTTON_COLOR = (31, 106, 165)
BUTTON_HOVER = (20, 72, 112)
WARN_COLOR = (231, 76, 60)

font_title = pygame.font.SysFont("segoeui", 32, bold=True)
font_label = pygame.font.SysFont("segoeui", 16)
font_input = pygame.font.SysFont("segoeui", 18)
font_info = pygame.font.SysFont("segoeui", 14, italic=True)
font_bold = pygame.font.SysFont("segoeui", 16, bold=True)

font_result_lbl = pygame.font.SysFont("segoeui", 20, bold=True)
font_result_val = pygame.font.SysFont("segoeui", 22, bold=True)
font_grade = pygame.font.SysFont("segoeui", 36, bold=True)

GRADE_COLORS = {
    "A": (46, 204, 113), # #2ecc71
    "B": (52, 152, 219), # #3498db
    "C": (241, 196, 15), # #f1c40f
    "D": (230, 126, 34), # #e67e22
    "F": (231, 76, 60) # #e74c3c
}

# State
math_text = ""
```

```

science_text = ""
english_text = ""
active_input = None # 0: math, 1: science, 2: english

error_message = ""
total_result = "0.0 / 300"
percentage_result = "0.00%"
grade_result = "-"
grade_color = TEXT_COLOR

def calculate():
    global error_message, total_result, percentage_result, grade_result, grade_color
    try:
        error_message = ""
        m = float(math_text)
        s = float(science_text)
        e = float(english_text)

        if any(x < 0 or x > 100 for x in [m, s, e]):
            error_message = "Marks must be between 0 and 100."
            return

        total = m + s + e
        perc = (total / 300.0) * 100.0

        if perc >= 90:
            grade = "A"
        elif perc >= 80:
            grade = "B"
        elif perc >= 70:
            grade = "C"
        elif perc >= 60:
            grade = "D"
        else:
            grade = "F"

        total_result = f"{total:.1f} / 300"
        percentage_result = f"{perc:.2f}%"
        grade_result = grade
        grade_color = GRADE_COLORS[grade]

    except ValueError:
        error_message = "Please enter valid numbers."

# Main loop
clock = pygame.time.Clock()

```

```

input_rects = [
    pygame.Rect(50, 140, 250, 35),
    pygame.Rect(50, 220, 250, 35),
    pygame.Rect(50, 300, 250, 35)
]
button_rect = pygame.Rect(75, 370, 200, 40)

running = True
while running:
    mouse_pos = pygame.mouse.get_pos()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                active_input = None
                for i, rect in enumerate(input_rects):
                    if rect.collidepoint(mouse_pos):
                        active_input = i

                if button_rect.collidepoint(mouse_pos):
                    calculate()

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_RETURN:
                    calculate()
                elif event.key == pygame.K_TAB:
                    if active_input is not None:
                        active_input = (active_input + 1) % 3
                    else:
                        active_input = 0
                elif active_input is not None:
                    if event.key == pygame.K_BACKSPACE:
                        if active_input == 0:
                            math_text = math_text[:-1]
                        elif active_input == 1:
                            science_text = science_text[:-1]
                        elif active_input == 2:
                            english_text = english_text[:-1]
                    else:
                        char = event.unicode
                        if char.isdigit() or char == ' ':
                            if active_input == 0:
                                math_text += char

```



```

        elif active_input == 1:
            science_text += char
        elif active_input == 2:
            english_text += char

screen.fill(BG_COLOR)

# Draw Title
title_surface = font_title.render("Marks Calculator", True, TEXT_COLOR)
screen.blit(title_surface, (WIDTH//2 - title_surface.get_width()//2, 20))

info_surface = font_info.render("(Enter marks out of 100 for each subject)", True,
GRAY)
screen.blit(info_surface, (WIDTH//2 - info_surface.get_width()//2, 55))

# Draw Inputs
labels = ["Math Marks:", "Science Marks:", "English Marks:"]
texts = [math_text, science_text, english_text]

for i in range(3):
    label_surf = font_label.render(labels[i], True, TEXT_COLOR)
    screen.blit(label_surf, (input_rects[i].x, input_rects[i].y - 25))

    color = INPUT_BG_ACTIVE if active_input == i else INPUT_BG_INACTIVE
    pygame.draw.rect(screen, color, input_rects[i], border_radius=5)

    # Border
    border_color = BUTTON_COLOR if active_input == i else GRAY
    pygame.draw.rect(screen, border_color, input_rects[i], 2, border_radius=5)

    # Text
    txt_surf = font_input.render(texts[i], True, TEXT_COLOR)
    if not texts[i] and active_input != i:
        ph_surf = font_input.render("max 100", True, GRAY)
        screen.blit(ph_surf, (input_rects[i].x + input_rects[i].width//2 -
ph_surf.get_width()//2, input_rects[i].y + 7))
    else:
        screen.blit(txt_surf, (input_rects[i].x + input_rects[i].width//2 -
txt_surf.get_width()//2, input_rects[i].y + 7))

# Draw Button
btn_color = BUTTON_HOVER if button_rect.collidepoint(mouse_pos) else
BUTTON_COLOR
pygame.draw.rect(screen, btn_color, button_rect, border_radius=8)
btn_txt = font_bold.render("Calculate Results", True, TEXT_COLOR)

```

```
screen.blit(btn_txt, (button_rect.x + button_rect.width//2 - btn_txt.get_width()//2,
button_rect.y + 10))
```

```
# Draw Error Message
```

```
if error_message:
```

```
    err_surf = font_label.render(error_message, True, WARN_COLOR)
```

```
    screen.blit(err_surf, (WIDTH//2 - err_surf.get_width()//2, 420))
```

```
# Draw Results
```

```
res_y = 440
```

```
# Total
```

```
tot_lbl = font_result_lbl.render("Total:", True, TEXT_COLOR)
```

```
tot_val = font_result_val.render(total_result, True, TEXT_COLOR)
```

```
screen.blit(tot_lbl, (50, res_y))
```

```
screen.blit(tot_val, (190, res_y - 2))
```

```
# Percentage
```

```
perc_lbl = font_result_lbl.render("Percentage:", True, TEXT_COLOR)
```

```
perc_val = font_result_val.render(percentage_result, True, TEXT_COLOR)
```

```
screen.blit(perc_lbl, (50, res_y + 40))
```

```
screen.blit(perc_val, (190, res_y + 38))
```

```
# Grade
```

```
grd_lbl = font_result_lbl.render("Grade:", True, TEXT_COLOR)
```

```
grd_val = font_grade.render(grade_result, True, grade_color)
```

```
screen.blit(grd_lbl, (50, res_y + 80))
```

```
screen.blit(grd_val, (190, res_y + 70))
```

```
pygame.display.flip()
```

```
clock.tick(60)
```

```
pygame.quit()
```

```
sys.exit()
```